

CONNECTING THE DOTS



ISC

High Performance

Basic Use cases: Monitoring and static optimization

Julita Corbalan (julita.corbalan@eas4dc.com, julita.corbalan@bsc.es)
Benjamin Czaja (benjamin.czaja@surf.nl)
Martijn Kruijer (martijn.kruijer@surf.nl)

Monitoring and application characterization

```
[julitac@int5 ~]$ eacct -j 5687690
```

JOB-STEP	USER	APPLICATION	POLICY	NODES	AVG/DEF/IMC(GHz)	TIME(s)	POWER(W)	GBS	CPI	ENERGY(J)	GFLOPS/W	IO(MBs)	MPI%	G-POW
(T/U)	G-FREQ	G-UTIL(G/MEM)												
5687690-sb	julitac	run_tensor.sh	NP	1	2.43/2.40/---	2612.00	448.98	---	---	1172725	---	---	---	---
5687690-8	julitac	DenseNet121_disa	MO	1	2.38/2.40/2.19	358.55	897.17	2.03	0.66	321685	0.0000	0.1	0.0	257.92 /257.92 1.410 92%/44%
5687690-7	julitac	DenseNet121_mixe	MO	1	2.38/2.40/2.19	189.30	876.59	0.82	0.67	165936	0.0000	0.2	0.0	238.88 /238.88 1.410 80%/52%
5687690-6	julitac	DenseNet121	MO	1	2.38/2.40/2.19	249.38	890.48	0.59	0.66	222070	0.0000	0.1	0.0	251.56 /251.56 1.410 88%/73%
5687690-5	julitac	VGG19_disable-tf	MO	1	2.38/2.40/2.19	646.86	877.18	2.33	0.52	567419	0.0000	0.1	0.0	238.58 /238.58 1.410 98%/26%
5687690-4	julitac	VGG19_mixed	MO	1	2.38/2.40/2.19	248.40	897.64	0.52	0.67	222972	0.0000	0.1	0.0	257.04 /257.04 1.410 96%/51%
5687690-3	julitac	VGG19	MO	1	2.38/2.40/2.19	261.37	907.44	3.21	0.61	237176	0.0000	0.1	0.0	267.94 /267.94 1.410 96%/59%
5687690-2	julitac	ResNet50_disable	MO	1	2.38/2.40/2.19	302.42	920.38	4.22	0.65	278338	0.0000	0.1	0.0	279.54 /279.54 1.410 94%/43%
5687690-1	julitac	ResNet50_mixed	MO	1	2.38/2.40/2.19	149.26	873.35	0.80	0.66	130356	0.0000	0.2	0.0	233.60 /233.60 1.403 84%/50%
5687690-0	julitac	ResNet50	MO	1	2.38/2.40/2.19	196.33	904.09	0.62	0.65	177504	0.0000	0.2	0.0	261.45 /261.45 1.372 87%/70%

Job Energy Efficiency metric: CPU jobs

- **CPU**

- **CPU Bound phases**

- Very low CPI , Cycles per instruction, (less than 0.5)
 - Low Memory bandwidth (depends on the architecture)
 - Percentage of MPI influences the CPI (MPI waitings are implemented with busy waiting, reduces the CPI)
 - High Gflops
 - Scale linearly with CPU frequency. Not too much opportunities for energy savings

- **Memory bound phases**

- Medium/High CPI (CPU has to wait for data → increases cycles per instructions)
 - High Memory bandwidth
 - Medium/High Gflops
 - Can take profit of reducing the CPU frequency. Do not scale linearly with CPU frequency



Job Energy Efficiency metric: CPU jobs

- PUE is the ratio of the total amount of power used by a computer data center facility to the power delivered to computing equipment
 - It's not a job metric!!
- Being energy efficient is not consuming less power, is doing an optimal utilization of the power we consume
- For CPU HPC applications, a traditional metric could be the GFLOPS/Watt
 - GFlops characterize the CPU activity
 - Watts the power consumption
- Not valid for
 - Data intensive jobs
 - Non-CPU intensive jobs

Job Energy Efficiency metric: GPU jobs

- GPU metrics reported by default (per-GPU)
 - GPU utilization
 - GPU memory utilization
 - GPU frequency
 - GPU power consumption
 - GPU flops (NVIDIA GPUS)
- GPU Utilization is not representative enough of the GPU activity
 - NVIDIA GPUs provides more metrics, but many of them are ratios (0...1), for example the FP activity
 - See next section to gather GPU extra metrics

Exercices

Monitoring and visualization

- Execution environment: Snellius
- Use cases and scripts in Snellius shared folder and Github
 - CPU use cases
 - GPU use cases
- Changing CPU frequency
 - CPU cases
- Energy efficiency vs Resource consumption
- Energy efficiency vs architecture

1- Basic EAR monitoring

- Codes in /projects/0/energy-course
- GIT: <https://github.com/sara-nl/ISC-EAR-tutorial/tree/main>
- Get the examples and test them
 - <https://github.com/sara-nl/ISC-EAR-tutorial/tree/main/scripts>
 - NPB CPU use cases
 - GROMACS-GPU
 - HemePure CPU
 - HemePure GPU
 - PyTorch
 - *Or select another use case from the list*
- Wait for jobs to finalize
- Get the ear metrics
 - module load ear
 - eacct -j jobid1
- Understand the metrics

For each case

- Execute with ear=on and get the metrics
 - Per job
 - Per node (if it applies)
 - Runtime metrics
- Understand application characteristics
 - Is my application CPU bound?
 - Is my application Memory bound?
 - Is it a power hungry application? Are we close to the TDP?
 - Does my application shows IO activity?
 - Does my application shows high MPI percentage?
 - Does my application present phases of execution?
- For GPU application
 - Is my application pure GPU? Or CPU/GPU ?
 - What is the GPU activity and power consumption? Are we close to the TDP?

eacct options

- `eacct -j jobid -l` → show average metrics (per-node)
- `eacct -j jobid -r` → shows runtime metrics (per-node)
- `eacct -j jobid -r c filename` → saves ear metrics in csv file

2- Static energy optimization

- What is the effect of changing my resource requirements? (Use GROMACS)
 - Number of Nodes
 - Number of tasks
 - Ratio tasks/cpus-per-task
 - Ratio tasks per GPU
- What is the effect of changing the CPU frequency? (use NPB)
 - EAR provides its own list of CPU frequencies, use `enode_info` to get the list
 - Is it worth to do it in CPU bound cases?
 - Is it worth to do it in Memory bound cases?

3- Energy efficient architectures

- GENOA nodes consumes more power than ROME nodes but....Are they more energy efficient (use NPB)
 - What is the total energy consumption for the same problem?
 - What's the ratio performance variation vs Energy/Power variation?

Go to ISC tutorial github and do the exercises

Access Snellius:

<https://ondemand.snellius.surf.nl/>

Github course material:

https://github.com/sara-nl/ISC-EAR-tutorial/tree/main/tutorials/monitoring_ear

CONNECTING THE DOTS



ISC

High Performance